Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики і програмної інженерії

Звіт

з дисципліни «Основи розробки програмного забезпечення на платформі Java»
Комп'ютерний практикум №1.
Основи програмування Java. Побудова класу

Виконав: гр. ІП-з41 Черепов Олександр Павлович

Перевірив: старший викладач Дифучина Олександра Юріївна

Оцінка: _____

Дата оцінки: _____

2026

Комп’ютерний практикум №1

Тема роботи: Основи програмування Java. Побудова класу.
Мета роботи: Дослідити механізм побудови класу в Java.

Завдання до роботи

1. Створити клас згідно з варіантом завдання. У головній програмі створити масив об’єктів класу та виконати тестові розрахункові дії з об’єктами з використанням методів класу. **30 балів.**
2. Дослідити можливість використання кількох конструкторів класу; можливість розробки класу без конструктора класу. Дослідити можливість визначення методу з одним іменем, але різним списком аргументів. Дослідити механізм інкапсуляції даних. **30 балів.**
3. Дослідити поняття статичного поля класу. Створити статичне поле для класу. **20 балів.**
4. Дослідити поняття статичного методу класу. Створити статичний метод та надати приклад його використання в головній програмі. **20 балів.**

Варіант: 6) Клас «співробітник».

Реалізація

Структура файлів:

```
programme/
├── employee/
│   ├── department
│   │   └── Department.java
│   ├── Employee.java
│   └── EmployeeSimple.java
└── Main.java
```

Лістинг програми:

programme/employee/departments/Department.java:

```
package employee.department;

public enum Department {
    PRODUCT,
    SALES,
    MARKETING,
    HR,
```

```
    PROCESS,
    CUSTOMER
  }
```

programme/employee/EmployeeSimple.java:

```
package employee;

import employee.department.Department;

// Task 2.2: Дослідити можливість розробки класу без конструктора класу
public class EmployeeSimple {
    private int id;
    private String name;
    private Department department;
    private double salary;

    public String getEmployeeData() {
        return "ID: " + id + ", Name: " + name + ", Department: " +
department + ", Salary: $" + salary;
    }

    public static void main(String[] args) {
        EmployeeSimple employee = new EmployeeSimple();
        System.out.println(employee.getEmployeeData());
    }
}
```

programme/employee/Employee.java:

```
package employee;
import employee.department.Department;

// Task 1.1: Створити клас згідно з варіантом завдання
public class Employee {
    // Task 3: Дослідити поняття статичного поля класу. Створити статичне
поле для класу
    private static int totalEmployees = 0;
    private static int nextId = 1;

    private int id;
    private String name;
    private Department department;
    private double salary;

    // Task 2.1: Дослідити можливість використання кількох конструкторів
класу
    public Employee() {
        id = nextId++;
        this.name = "Unknown";
```

```
        this.department = null;
        this.salary = 0;
        totalEmployees++;
    }

    public Employee(String name, Department department, double salary) {
        id = nextId++;
        this.name = name;
        this.department = department;
        this.salary = salary;
        totalEmployees++;
    }

    public int getId() {
        return id;
    }

    public String getEmployeeData() {
        return "ID: " + id + ", Name: " + name + ", Department: " +
department + ", Salary: $" + salary;
    }

    // Task 4.1: Дослідити поняття статичного методу класу
    public static int getTotalEmployees() {
        return totalEmployees;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("Name cannot be null or
empty.");
        } else if (name.length() > 100 || name.length() < 2) {
            throw new IllegalArgumentException("Name must be between 2 and
100 characters.");
        }

        this.name = name;
    }

    public Department getDepartment() {
        return department;
    }

    public void setDepartment(Department department) {
        if (department == null) {
            throw new IllegalArgumentException("Department cannot be null.");
        }
```

```
        this.department = department;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        if (salary < 0) {
            throw new IllegalArgumentException("Salary cannot be negative.");
        }
        this.salary = salary;
    }

    // Task 2.3: Дослідити можливість визначення методу з  одним іменем, але
різним списком аргументів
    public void giveRaise(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Raise amount cannot be
negative.");
        }
        this.salary += amount;
    }

    public void giveRaise(double percentage, boolean isPercentage) {
        if (isPercentage) {
            if (percentage < 0) {
                throw new IllegalArgumentException("Raise percentage cannot
be negative.");
            }

            double amount = this.salary * (percentage / 100);
            this.salary += amount;
        }
    }
}
```

programme/Main.java:

```
import java.util.Random;

import employee.Employee;
import employee.department.Department;

public class Main {
    private static Department[] departments = Department.values();
    static Random random = new Random();

    private static int getRandomDepartmentIndex() {
        return random.nextInt(departments.length);
    }
```

```
    private static void printEmployees(Employee[] employees) {
        for (int i=0; i<employees.length; i++) {
            System.out.println(i + 1 + ". " +
employees[i].getEmployeeData());
        }
    }

    private static double getAverageSalary(Employee[] employees) {
        double totalSalary = 0;
        for (Employee employee : employees) {
            totalSalary += employee.getSalary();
        }

        // Task 4.2: Створити статичний метод та надати приклад його
використання в головній програмі
        return totalSalary / Employee.getTotalEmployees();
    }

    private static Employee getHighestSalaryEmployee(Employee[] employees) {
        Employee topEmployee = employees[0];

        for (Employee employee : employees) {
            if (employee.getSalary() > topEmployee.getSalary()) {
                topEmployee = employee;
            }
        }
        return topEmployee;
    }
    public static void main(String[] args) {
        // Task 1.2: У головній програмі створити масив об'єктів класу
        Employee[] employees = new Employee[10];

        System.out.println("--- BEFORE RAISE ---");
        for (int i=0; i<10; i++) {
            Employee employee = new Employee();

            employee.setName("Employee " + (i + 1));
            employee.setDepartment(departments[getRandomDepartmentIndex()]);
            employee.setSalary(Math.floor(Math.random() * 10000));

            employees[i] = employee;
        }

        printEmployees(employees);

        // Task 1.3: Виконати тестові розрахункові дії з об'єктами з
використанням методів класу
        System.out.println("Total Employees: " +
Employee.getTotalEmployees());
```

```
        System.out.println("Average Salary: $" +
getAverageSalary(employees));
        System.out.println("Employee with the highest salary: " +
getHighestSalaryEmployee(employees).getName() + ";  $" +
getHighestSalaryEmployee(employees).getSalary());

        System.out.println("--- AFTER RAISE ---");
        for (int i=0; i<Employee.getTotalEmployees()-1; i++) {
            int probRaise = random.nextInt(2);

            if (probRaise == 1) {
                System.out.println(employees[i].getName() + " got a raise!");
                int probPercent = random.nextInt(2);

                if (probPercent == 1) {
                    employees[i].giveRaise(Math.floor(Math.random() * 100),
true);
                } else {
                    employees[i].giveRaise(Math.floor(Math.random() * 1000));
                }
                ;
            }
        }

        printEmployees(employees);

        System.out.println("Average Salary: $" +
getAverageSalary(employees));
        System.out.println("Employee with the highest salary: " +
getHighestSalaryEmployee(employees).getName() + ";  $" +
getHighestSalaryEmployee(employees).getSalary());
    }
}
```

Результати тестування програми:

```
--- BEFORE RAISE ---
1. ID: 1, Name: Employee 1, Department: PRODUCT, Salary: $8546.0
2. ID: 2, Name: Employee 2, Department: CUSTOMER, Salary: $708.0
3. ID: 3, Name: Employee 3, Department: CUSTOMER, Salary: $7892.0
4. ID: 4, Name: Employee 4, Department: HR, Salary: $2198.0
5. ID: 5, Name: Employee 5, Department: SALES, Salary: $6271.0
6. ID: 6, Name: Employee 6, Department: MARKETING, Salary: $4453.0
7. ID: 7, Name: Employee 7, Department: HR, Salary: $1085.0
8. ID: 8, Name: Employee 8, Department: MARKETING, Salary: $7957.0
9. ID: 9, Name: Employee 9, Department: CUSTOMER, Salary: $4788.0
10. ID: 10, Name: Employee 10, Department: SALES, Salary: $2625.0
Total Employees: 10
Average Salary: $4652.3
Employee with the highest salary: Employee 1;  $8546.0
--- AFTER RAISE ---
Employee 1 got a raise!
Employee 4 got a raise!
Employee 5 got a raise!
Employee 6 got a raise!
Employee 7 got a raise!
Employee 8 got a raise!
1. ID: 1, Name: Employee 1, Department: PRODUCT, Salary: $9196.0
2. ID: 2, Name: Employee 2, Department: CUSTOMER, Salary: $708.0
3. ID: 3, Name: Employee 3, Department: CUSTOMER, Salary: $7892.0
4. ID: 4, Name: Employee 4, Department: HR, Salary: $3231.06
5. ID: 5, Name: Employee 5, Department: SALES, Salary: $11538.64
6. ID: 6, Name: Employee 6, Department: MARKETING, Salary: $5127.0
7. ID: 7, Name: Employee 7, Department: HR, Salary: $2008.0
8. ID: 8, Name: Employee 8, Department: MARKETING, Salary: $8911.0
9. ID: 9, Name: Employee 9, Department: CUSTOMER, Salary: $4788.0
10. ID: 10, Name: Employee 10, Department: SALES, Salary: $2625.0
Average Salary: $5602.469999999999
Employee with the highest salary: Employee 5;  $11538.64
PS C:\archive\EducationalWork\KPI\Java\КП1\programme>
```

Висновок

Під час виконання комп'ютерного практикуму було успішно досліджено механізм побудови класу в Java, реалізовано клас «співробітник» та виконано розрахункові дії з масивом об'єктів. Також на практиці застосовано механізми інкапсуляції та статичні поля і методи.

Головними перевагами використання класів є надійний захист даних від зовнішнього втручання завдяки інкапсуляції, висока структурованість коду, а також зручність його повторного використання через створення безлічі об'єктів за єдиним шаблоном. Гнучкість розробки додатково підвищується завдяки можливості використання кількох конструкторів та перевантаження методів.

До недоліків використання об'єктно-орієнтованого підходу можна віднести складність перевикористання класів з інших проектів через сильні зв’язки шляхом наслідування, а також дещо більші витрати оперативної пам'яті та системних ресурсів на створення і зберігання об'єктів порівняно з традиційним процедурним програмуванням.

Контрольні запитання

1. ***З яких елементів складається клас?***

   Складовими частинами класу є поля, в яких зберігається інформація про дані, методи, які призначені для обробки даних, та конструктори, що призначені для створення екземпляру класу.

2. ***У чому полягає інкапсуляція даних?***

   Інкапсуляція полягає у обмеженні доступу до даних. Елемети класу можуть позначатися як *public* – доступні ззовні класу, *private* – доступні лише всередині класу, *protected* – доступні всередині класу та для класів-нащадків.

3. ***Чим відрізнається структура даних від класу?***

   Клас містить в собі дані та методи для їх обробки, тоді як структури даних служать лише для їх зберігання.

4. ***Що таке конструктор класу?***

   Конструктор класу це метод, призначений для ініціалізації полів даних об’єкта та створення екземпляру класу.

5. ***Як створити екземпляр класу?***

   За допомогою ключового слова *new* викликати конструктор та передати відповідні агрументи (якщо необхідно).

6. ***Що є об’єкт в термінології ООП?***

   Об’єкт в ООП – це екземпляр певного класу, який має притаманні тільки йому унікальні дані.

7. ***Що таке модифікатор доступу?***

   Це ключове слово, яке визначає рівень доступу до полів або методів класу з інших частин програми.

8. ***Які типи даних використовуються мовою програмування Java?***

   Java використовує базові примітивні типи (наприклад, *int, double, char, boolean*) та посилальні типи (класи, масиви, наприклад *String*).

9. ***Що таке метод доступу? Як розробити метод доступу?***

   Це метод, розроблений для безпечного доступу до приватних даних з інших класів. Їх розробляють за стандартом: *getField()* для отримання значення та *setField(...)* для його зміни.

10. ***Що означає термін «інкапсуляція»?***

    Це обмеження прямого доступу до даних об'єкта. Доступ до цих даних дозволяється тільки через прописані методи.

11. Чим відрізняється ОО програма від процедурної програми?

ОО програма базується на об'єктах і використовує принципи ООП (інкапсуляція, спадкування та поліморфізм). Вона об'єднує дані та методи їх обробки в класи, тоді як процедурна програма розділяє дані та функції.

12. Як розробити конструктор класу?

Треба створити метод із модифікатором доступу, назва якого точно співпадає з назвою класу, та вказати перелік параметрів у дужках (який може бути порожнім).

13. Чи може бути конструктор класу з модифікатором private?

Так, може. Однак такий підхід забороняє створення екземплярів цього класу ззовні (наприклад, для класів, що містять лише статичні математичні методи).

14. Що є екземпляром класу? Як створити екземпляр класу?

Екземпляр класу – це створений об'єкт. Він створюється за допомогою ключового слова *new* та виклику конструктора класу.

15. Чим відрізняється статичний метод від звичайного?

Статичний метод притаманний всьому класу, тоді як звичайний метод стосується конкретного об’єкту. Тому статичний метод можна викликати навіть тоді, коли не створено жодного екземпляру класу.

16. Що означає модифікатор static?

Він означає, що поле або метод належить самому класу, а не окремим об'єктам. Статичне поле існує в єдиному екземплярі і є спільним для всіх об’єктів цього класу.